

Tutorial - Semana 5, Encontro 2

Introdução

No Encontro 1, o projeto ganhou o contrato `Interactable`, implementado pela Scene `Door.tscn`, e um `InteractionComponent` no Player capaz de detectar objetos próximos via `Area3D` e chamar `interact()` sem conhecer o tipo concreto do objeto. Este encontro completa o par que vai sustentar todo objeto interativo do Vertical Slice: os **Signals**. Em vez de a Door apenas imprimir uma mensagem dentro de `interact()`, ela passa a emitir um Signal, permitindo que qualquer outro sistema reaja ao evento sem que a Door precise conhecê-lo.

Este tutorial dá continuidade direta ao Encontro 1 — o contrato `Interactable`, a Scene `Door.tscn` e o `InteractionComponent` já devem existir antes de começar.

Objetivos da semana

- Explicar Signals como padrão observer para reação a eventos de interação.
- Diferenciar chamada direta de método de emissão de Signal.
- Implementar um Signal disparado por interação e conectá-lo a uma reação concreta em uma Scene do grupo.

Resultado esperado ao final da semana

Ao final da Semana 5 (Encontros 1 e 2), cada estudante terá um contrato `Interactable` implementado por ao menos uma Scene do Vertical Slice, com um Signal disparado por interação e conectado a uma reação concreta. Este tutorial cobre apenas o **Encontro 2**: a adição do Signal à Door, a conexão a uma reação visível, e o desafio de cada grupo construir seu próprio objeto interativo com mecanismo de acionamento livre.

Pré-requisitos

- Contrato `Interactable`, Scene `Door.tscn` e `InteractionComponent` do Encontro 1, já testados (ver Tutorial - Semana 5, Encontro 1).
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto do Encontro 1, com `Door.tscn` reagindo a `interact()` e o `InteractionComponent` do Player funcionando com pelo menos duas instâncias de teste.

Arquivos necessários

- Nenhum arquivo externo adicional.

Assets utilizados

- Opcionalmente, um segundo mesh simples para um objeto interativo alternativo (alavanca), caso o grupo escolha essa variação no desafio final.

Projeto esperado

- Projeto aberto no Godot 4.7, com o contrato `Interactable` e o `InteractionComponent` do Encontro 1 já testados.
- Área de teste no nível existente com espaço para pelo menos dois objetos interativos distintos.

Imagem sugerida

Objetivo: mostrar o painel Node do Godot com o Signal `interacted` declarado na Scene `Door.tscn` e sua conexão a uma função de reação. Enquadramento: captura de tela da aba **Node > Signals** do editor Godot, com `Door.tscn` aberta. Elementos importantes: lista de Signals do nó, o Signal `interacted` customizado, o ícone de conexão ativa. Destaque: o Signal `interacted` e a seta indicando a função conectada. Legenda sugerida: "Signal `interacted` declarado na Door e conectado à função de reação."

Parte 1 — Signals como padrão observer

Objetivo

Entender por que a Door não deve chamar diretamente a função que abre a porta, e sim emitir um aviso de que foi interagida.

Conceito

O contrato `Interactable` resolve "como um objeto responde a uma chamada"; Signals resolvem um problema complementar: "como um objeto avisa que algo aconteceu, sem saber quem está ouvindo". Esse é o padrão **observer**, presente em praticamente toda engine de jogos moderna. No Godot, um Signal é declarado na Scene emissora e conectado — via editor ou código — a uma ou mais funções de reação em outras Scenes, sem que o emissor precise conhecer o receptor.

Combinado ao contrato `Interactable` construído no Encontro 1, isso permite que `Door.interact()` apenas emita o aviso "fui interagida", e qualquer parte do projeto — a própria Door, um som, uma animação, futuramente o `GameManager` — reaja a esse aviso de forma independente. É a mesma separação que sustentará a ampliação da interação conectada ao Inventário na Semana 10.

Passo a passo

Esta parte não possui etapas de implementação no editor — é a base conceitual antes da Parte 2.

1. Revisar com a turma o funcionamento de `interact()` e do `InteractionComponent` do Encontro 1.
2. Perguntar: "se três sistemas diferentes precisassem reagir à interação com a porta (abrir a porta, tocar um som, atualizar o `GameManager`), a função `interact()` deveria chamar os três diretamente?"
3. Concluir que a resposta correta é não — `interact()` deve apenas emitir um Signal, e cada sistema interessado se conecta a ele de forma independente.

Resultado esperado

A turma entende Signals como mecanismo para avisar sem conhecer quem reage, distinto da chamada direta de função usada em `interact()`.

Verificando

1. Confirmar que os estudantes conseguem diferenciar, em suas próprias palavras, "chamar uma função" de "emitir um Signal".

Problemas comuns

- Achar que Signal e função são a mesma coisa com nomes diferentes: reforçar que o emissor de um Signal não precisa saber quantas (ou se alguma) função está conectada a ele; uma chamada direta sempre precisa de uma referência ao receptor.

Boas práticas

- Manter essa discussão breve, reservando a maior parte do encontro para a implementação nas Partes 2 e 3.

Comparação com Unity

A Unity resolve o mesmo padrão observer com `UnityEvent` (configurável pelo Inspector, semelhante à conexão de Signal pelo editor do Godot) ou com `event / Action` de C# (mais próximo da conexão via código). A diferença relevante não é a sintaxe, mas o fato de o Godot tratar Signals como um mecanismo de primeira classe do editor — declarado, listado e conectado visualmente em qualquer Node — enquanto na Unity o mesmo resultado depende de qual das duas abordagens a equipe escolheu adotar como convenção própria.

Parte 2 — Declarando e conectando o Signal da Door

Objetivo

Substituir o `print()` de `interact()` por um Signal, e conectar esse Signal a uma reação visível.

Conceito

Um Signal customizado é declarado no topo do script com a palavra-chave `signal`, seguindo a convenção de nomenclatura do projeto — `snake_case`, verbo no passado ou substantivo de evento (por exemplo, `interacted`), conforme o `PROJECT_ARCHITECTURE.md` (seção 9). Emitir o Signal dentro de `interact()` avisa que a interação ocorreu; a reação (abrir a porta) passa a viver em uma função separada, conectada ao Signal — nunca dentro da própria `interact()`.

Passo a passo

1. Abra o script `door.gd`, criado no Encontro 1.
2. No topo do script, logo abaixo de `class_name Door`, declare o Signal: `signal interacted`.
3. Substitua o corpo de `interact()`, removendo o `print()` e emitindo o Signal:

```
func interact() -> void:
    interacted.emit()
```

4. Declare uma variável de estado simples para controlar a reação visual, por exemplo `var aberta: bool = false`.
5. Crie uma função de reação separada, por exemplo `_abrir_fechar()`, que alterna `aberta` e modifica visualmente a Door (por exemplo, alterando a rotação do `MeshInstance3D` em 90 graus, ou trocando sua cor via `MaterialOverride`).
6. Na aba **Node** do editor, com a Scene `Door.tscn` aberta e o nó raiz `Door` selecionado, abra a subaba **Signals**.
7. Localize o Signal customizado `interacted` na lista e conecte-o à função `_abrir_fechar()` do próprio nó `Door` (conexão a si mesmo é válida e comum para reações locais).
8. Salve o script e a Scene (**Ctrl+S**).

Resultado esperado

Ao chamar `interact()`, a Door emite o Signal `interacted`, que dispara `_abrir_fechar()`, produzindo uma mudança visual visível (rotação ou cor) — sem que `interact()` chame `_abrir_fechar()` diretamente.

Verificando

1. Rode `level_exploration.tscn`, aproxime o Player da Door e pressione **E** (ação `interagir` do Encontro 1).
2. Confirme que a Door muda visualmente (abre/fecha) a cada interação.
3. Abra a subaba **Signals** da Door no editor e confirme que `interacted` aparece conectado a `_abrir_fechar()`, com o ícone de conexão ativa.

Problemas comuns

- Chamar `_abrir_fechar()` diretamente dentro de `interact()`, além de emitir o Signal: isso reintroduz a chamada direta que o Signal existe para evitar — `interact()` deve apenas emitir, nunca também chamar a reação diretamente.
- Esquecer de conectar o Signal pela subaba **Signals** após declará-lo no script: um Signal declarado mas não conectado é emitido sem produzir nenhum efeito visível.
- Nomear o Signal fora da convenção do projeto (por exemplo, `OnInteract` em vez de `interacted`): manter snake_case e verbo no passado/substantivo de evento, conforme a seção 9 do PROJECT_ARCHITECTURE.md.

Boas práticas

- Manter a lógica de reação (`_abrir_fechar()`) sempre em uma função separada da função do contrato (`interact()`), mesmo quando ambas estão na mesma Scene.
- Preferir nomes de Signal no padrão `substantivo_verbo_passado` (`interacted`, `item_collected`), reutilizável em qualquer objeto interativo futuro.

Comparação com Unity

Na Unity, o mesmo resultado exigiria declarar um campo `public UnityEvent OnInteracted;` (conectável pelo Inspector, de forma parecida com a subaba Signals do Godot) ou um `event Action OnInteracted;` (conectado via código, em `OnEnable / Start`). O comportamento é equivalente — emitir sem conhecer o receptor —, mas a Unity exige escolher entre as duas abordagens, enquanto o Godot oferece Signals como único mecanismo nativo, disponível tanto pelo editor quanto por código.

Parte 3 — Desafio: objeto interativo próprio do grupo

Objetivo

Aplicar contrato `Interactable` + Signal a um objeto interativo distinto da Door demonstrada, com mecanismo de acionamento escolhido pelo grupo.

Conceito

A combinação contrato + Signal construída nas Partes 1 e 2 não é exclusiva da Door: qualquer Scene que implemente `interact()` e emita um Signal próprio se encaixa no mesmo `InteractionComponent` do Player, sem exigir nenhuma alteração nele. Esse é o teste real do desacoplamento ensinado nesta semana — um segundo objeto interativo, com uma reação completamente diferente da Door, deve funcionar sem tocar em nenhum código já escrito no Encontro 1.

Passo a passo

1. Em grupo, escolha um objeto interativo diferente da Door (por exemplo, `Lever.tscn`, uma alavanca) e o mecanismo de acionamento (alavanca, chave, proximidade).
2. Crie a nova Scene em `scenes/interactables/`, seguindo a mesma estrutura da Door (nó raiz com collider, `MeshInstance3D`, script próprio com `class_name`).
3. No script da nova Scene, declare o Signal correspondente ao evento (por exemplo, `signal lever_pulled`) e implemente `interact()` emitindo esse Signal.
4. Implemente a função de reação (por exemplo, `_alternar_alavanca()`) e conecte-a ao Signal pela subaba **Signals**, como feito na Parte 2 com a Door.
5. Instancie a nova Scene em `level_exploration.tscn`, próxima à Door já existente.
6. Rode a Scene e teste a interação com os dois objetos (Door e o novo objeto do grupo), confirmando que o `InteractionComponent` do Player funciona com ambos sem nenhuma alteração.

Resultado esperado

Cada grupo possui, ao final da semana, pelo menos um objeto interativo funcional no Vertical Slice, distinto da Door, implementando o contrato `Interactable`, disparando um Signal próprio na

interação e reagindo a esse Signal com um comportamento visível.

Verificando

1. Aproxime o Player do novo objeto interativo do grupo e confirme a chamada de `interact()` (Output ou reação visual).
2. Confirme que o Signal customizado do novo objeto aparece conectado na subaba **Signals**.
3. Teste a interação alternando entre a Door e o novo objeto, sem reiniciar o projeto, confirmando que o `InteractionComponent` do Player não precisou de nenhuma alteração.

Problemas comuns

- Duplicar lógica do `InteractionComponent` dentro do novo objeto interativo: o `InteractionComponent` já existe no Player e deve ser reutilizado sem alteração — o novo objeto só precisa implementar o contrato e o Signal.
- Nomear a função de contrato de forma diferente de `interact()` no novo objeto: quebra a checagem `has_method("interact")` do `InteractionComponent`, feita de forma idêntica para qualquer objeto interativo.

Boas práticas

- Revisar a Door como referência de estrutura (contrato + Signal + reação separada) antes de implementar o novo objeto, mas produzir uma reação visual própria, não uma cópia.
- Nomear o novo Signal de forma específica ao evento representado (`lever_pulled`, não `interacted` reaproveitado), evitando ambiguidade quando vários objetos interativos existirem no mesmo nível.

Comparação com Unity

O mesmo desafio, na Unity, exigiria implementar `IInteractable` na nova classe (`Lever : MonoBehaviour, IInteractable`) e declarar um novo `UnityEvent / event Action` próprio para o evento da alavanca — reaproveitando o mesmo `InteractionComponent` -equivalente do Player sem alteração, exatamente como no Godot. A escalabilidade do contrato é o ponto de comparação central desta etapa, não a sintaxe de cada engine.

Ao final da semana

Ao final da Semana 5, o projeto do Vertical Slice deve conter:

- O `GameManager` e o `SaveManager` da Semana 4, sem nenhuma alteração.
- O contrato `Interactable`, implementado pela Scene `Door.tscn`, com Signal `interacted` conectado a uma reação visível (Encontro 1 e Parte 2 do Encontro 2).
- O `InteractionComponent` do Player, detectando objetos interativos via `Area3D` e chamando `interact()` sem conhecer o tipo concreto do objeto.
- Um segundo objeto interativo próprio de cada grupo (por exemplo, `Lever.tscn`), implementando o mesmo contrato e um Signal próprio, funcionando com o `InteractionComponent` sem nenhuma alteração nele.

Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 2), este resultado corresponde à conclusão dos itens "Contrato `Interactable` (interface via duck typing)", "Signals de interação" e ao início do item "Door, Lever" do roadmap. O par contrato `Interactable` + Signal construído aqui é pré-requisito direto do `Checkpoint` da Semana 7 (contrato + persistência) e da ampliação da interação conectada ao Inventário na Semana 10.

Desafio

Cada grupo implementa um objeto interativo (porta ou equivalente escolhido pelo grupo) usando contrato `Interactable` + Signal, com liberdade de mecanismo de acionamento (alavanca, chave, proximidade) — este é o desafio central da Parte 3 e é avaliado por Feedback formal (Rubrica 2 — Desafios Técnicos: Solução proposta, Uso correto do Godot, Criatividade, Organização, Funcionamento).

Checklist

- ☐ Signal `interacted` declarado em `door.gd` e emitido dentro de `interact()`
- ☐ Função de reação separada (`_abrir_fechar()`) conectada ao Signal pela subaba Signals
- ☐ Reação visível da Door testada com sucesso via `InteractionComponent`
- ☐ Segundo objeto interativo do grupo criado, com contrato `Interactable` e Signal próprio
- ☐ `InteractionComponent` do Player funcionando com os dois objetos, sem alteração
- ☐ Feedback formal recebido sobre a solução de interação do grupo

Glossário

- **Signal:** mecanismo nativo do Godot para um Node avisar que um evento ocorreu, permitindo que qualquer outro Node se conecte e reaja, sem que o emissor conheça o receptor.
- **Padrão observer:** padrão de projeto em que um objeto (observável) notifica outros (observadores) sobre eventos, sem depender diretamente deles; no Godot, implementado nativamente via Signals.
- **interacted:** nome do Signal customizado emitido pela Door ao ser interagida, seguindo a convenção de nomenclatura do projeto (snake_case, verbo no passado).

Referências

- Godot Documentation — Signals:
https://docs.godotengine.org/en/stable/getting_started/step_by_step/signals.html
- Godot Documentation — GDScript (métodos, `has_method`):
<https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/index.html>
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa) — Events:
<https://docs.unity3d.com/Manual/UnityEvents.html>
- Canais recomendados para consulta complementar (não substituem a documentação oficial):
GDQuest (<https://www.youtube.com/@GDQuest>), Clear Code
(<https://www.youtube.com/@ClearCode>)